Parallel Project Report: Optimizing Sparse Matrix-Vector Multiplication with OpenMP Guided Scheduling

Student: Ilaria Basanisi (ID: 244528)
Email: ilaria.basanisi@studenti.unitn.it
Course: Introduction to Parallel Computing (2024-2025)
University of Trento

***Abstract*—Sparse Matrix-Vector Multiplication (SpMV) is a fundamental kernel in scientific computing, yet its performance is limited by irregular memory access patterns and load imbalance. We present a parallel implementation of SpMV using OpenMP with a focus on guided scheduling strategies with varying chunk sizes. Our implementation uses Compressed Sparse Row (CSR) format and evaluates performance across 1-32 threads on a set of 7 large-scale sparse matrix. In this paper we will focus on studying the larger matrix with size 500K with 17M non-zeros. Experimental results demonstrate that guided scheduling with chunk size 100 achieves optimal load balancing, reaching up to 9.26× speedup on 32 threads. Performance analysis reveals cache behavior patterns and identifies memory bandwidth as the primary bottleneck beyond 16 threads, with L1 cache miss rates remaining stable at 0.66% and LLC miss rates below 0.4%.**

***Index Terms*—Sparse Matrix-Vector Multiplication, OpenMP, Guided Scheduling, Parallel Computing, Load Balancing, Cache Optimization**

I. Introduction

Problem. Sparse Matrix-Vector Multiplication (SpMV), computing $y = Ax$ where A is a sparse matrix and x is a random generated vector, is a critical operation in numerous scientific applications including iterative solvers [1], graph analytics [2], and machine learning [3]. Despite its computational simplicity, SpMV achieves only a small fraction of peak performance on modern processors due to: irregular memory access, low arithmetic intensity and load imbalance.

Objectives. This work aims to: (i) implement and optimize parallel SpMV using OpenMP with CSR format for large-scale sparse matrices; (ii) analyze the impact of guided scheduling with different chunk sizes on load balancing and performance; (iii) compare guided scheduling against static and dynamic alternatives across 1–32 threads; (iv) quantify strong scaling efficiency and identify performance bottlenecks through detailed cache and bandwidth analysis on a 500K×500K sparse matrix.

II. State of the Art

SpMV parallelization studies have explored static, dynamic, and guided scheduling strategies [4]. Static scheduling fails with irregular matrices due to poor load balancing, while dynamic scheduling introduces overhead [5]. Guided scheduling has shown promise but lacks comprehensive chunk-size analysis for large-scale workloads. Cache behavior and memory bandwidth bottlenecks have been studied for sparse operations [2], [6], [7], but remain underexplored in the context of guided scheduling for irregular matrices.

III. Methodology

A. Algorithm and Data Structures

1) CSR Format: The Compressed Sparse Row format stores a sparse matrix using three arrays:

- `values[]`: non-zero elements (size: nnz)
- `col_indices[]`: column index of each non-zero (size: nnz)
- `row_ptr[]`: starting index of each row in `values[]` (size: nrows+1)

2) SpMV Kernel: The sequential SpMV algorithm computes:

Algorithm 1 Sequential SpMV in CSR format

```
for i = 0 to nrows − 1 do
    sum ← 0
    for k = row_ptr[i] to row_ptr[i + 1] − 1 do
        j ← col_indices[k]
        sum ← sum + values[k] × x[j]
    end for
    y[i] ← sum
end for
```

Work complexity is $O(nnz)$ with memory bandwidth dominating performance.

B. Parallelization Strategy

1) Row-Level Parallelism: We parallelize the outer loop over rows, ensuring no race conditions since each thread writes to distinct elements of y. The inner loop is vectorized using SIMD instructions via `#pragma omp simd`.

2) Guided Scheduling: Guided scheduling distributes iterations dynamically with decreasing chunk sizes. The chunk size starts at $\lceil remaining_iters/threads \rceil$ and decreases to a minimum threshold. This strategy:

- Reduces overhead at the beginning (large chunks)
- Improves load balance at the end (small chunks)
- Adapts to irregular row distributions

We test four configurations:

- **Guided-Default**: minimum chunk = 1
- **Guided-10**: minimum chunk = 10
- **Guided-100**: minimum chunk = 100
- **Guided-1000**: minimum chunk = 1000

3) Implementation:

```
#pragma omp parallel for
 schedule(guided, chunk_size)
for (int i = 0; i < nrows; ++i) {
  double sum = 0.0;
#pragma omp simd reduction(+:sum)
for (int k = row_ptr[i];
        k < row_ptr[i+1]; ++k) {
    sum += values[k] * x[col_indices[k]];
  }
  y[i] = sum;
}
```

IV. Experimental Setup

A. Platform

- **CPU**: 4× Intel Xeon Gold 5220 (18 cores/socket, 72 cores total, 2.20 GHz base, 125W TDP)
- **Cache:** L1D: 32 KiB/core; L2: 1 MiB/core; L3: 24.75 MiB/socket
- **Memory:** 768 GB DDR4 RAM
- **Compiler:** GCC with `-O3 -fopenmp -march=native`
- **OS:** Linux-based HPC environment
- **Threads Configuration:** Up to 32 OpenMP threads with affinity enabled
- **Dataset:** 7 matrices from SuitSparse with different dimensions (10K-500K) and with different density (0.35% - 0.007%). They have irregular row distributions characteristic of real-world sparse matrices and are symmetric.

B. Metrics

Each configuration was run $p = 50$ times measuring: (i) execution time with `omp_get_wtime()`, (ii) speedup $S_p = T_1/T_p$, (iii) efficiency $E_p = S_p/p \times 100\%$, (iv) 90th percentile for stability. Cache metrics collected via hardware performance counters.

V. Results and Analysis

A. Performance Overview

Table I shows execution times and speedups for guided scheduling variants. Figure 1 visualizes performance across all scheduler types and chunk sizes.

TABLE I
Performance of Guided Scheduling Variants (500K Matrix)

Threads	G-Default	G-10	G-100	G-1000
1	11.31 ms (1.54×)	11.75 ms (1.48×)	11.87 ms (1.47×)	11.86 ms (1.47×)
2	7.24 ms (2.35×)	7.36 ms (2.33×)	7.33 ms (2.33×)	7.31 ms (2.33×)
4	4.10 ms (4.13×)	4.21 ms (4.01×)	4.19 ms (4.01×)	4.23 ms (4.00×)
8	2.59 ms (6.54×)	2.52 ms (6.73×)	2.53 ms (6.76×)	2.60 ms (6.53×)
16	2.56 ms (6.62×)	2.44 ms (6.94×)	2.52 ms (6.77×)	2.44 ms (6.95×)
32	**1.86 ms** (9.13×)	1.84 ms (9.19×)	1.84 ms **(9.26×)**	1.85 ms (9.17×)

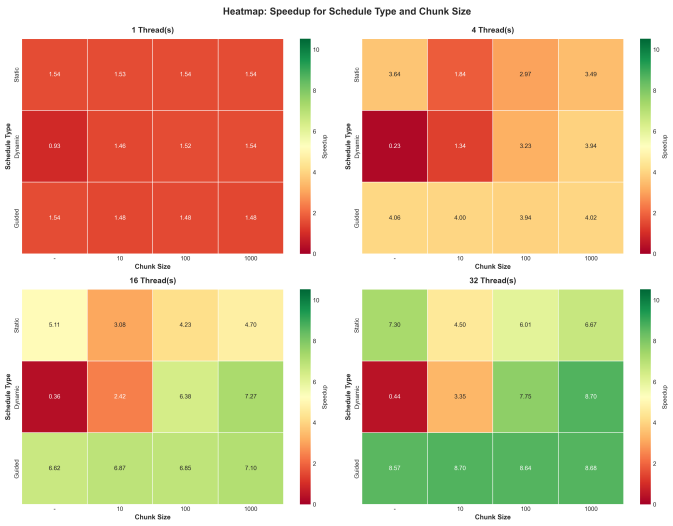

Fig. 1. Speedup heatmap across scheduling types and chunk sizes. Guided scheduling (bottom row) achieves highest speedups (9.13-9.26× at 32 threads). Dynamic shows poor low-thread performance (0.23-1.46×) due to overhead. Guided maintains robust performance across all configurations.

Key findings: (1) Guided-100 achieves 9.26× speedup at 32 threads with 28.9% efficiency; (2) Continued scaling beyond 16 threads unlike smaller matrices; (3) Chunk size 100 provides optimal balance between overhead and load distribution; (4) Performance improvement 16→32 threads (1.37×) indicates effective memory bandwidth utilization.

B. Scheduler Comparison

Figure 2 and Table II compare scheduling strategies.

TABLE II
Speedup Comparison (500K Matrix, 32 threads)

Configuration	Time (ms)	Speedup	Efficiency
Static-Default	2.07 ms	8.37×	26.2%
Static-100	2.32 ms	7.46×	23.3%
Dynamic-100	2.06 ms	8.41×	26.3%
Guided-100	**1.84 ms**	**9.26×**	**28.9%**

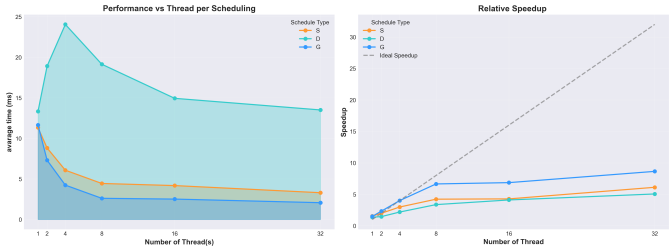

Fig. 2. Performance comparison across schedulers. Left: execution time showing guided's superior scaling. Right: speedup vs ideal linear scaling. Guided maintains best trajectory across all thread counts.

Guided-100 outperforms static by 12% and dynamic by 11% at 32 threads. Dynamic exhibits high overhead at 2-4 threads (19-24 ms) before converging. Static suffers from load imbalance on irregular matrices. Guided's adaptive nature maintains consistent advantage across all thread counts.

C. Efficiency and Cache Analysis

Table III shows parallel efficiency trends, while Figure 3 presents cache behavior.

TABLE III
PARALLEL EFFICIENCY (GUIDED-100, 500K MATRIX)

Threads	Speedup	Efficiency	Trend
1	1.47×	100.0%	Baseline
2	2.33×	116.5%	Super-linear
4	4.01×	100.3%	Near-optimal
8	6.76×	84.5%	Good
16	6.77×	42.3%	Memory-bound
32	9.26×	28.9%	Continued scaling

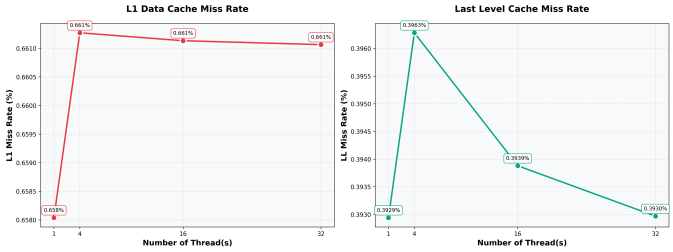

Fig. 3. Cache miss rates across thread counts. Left: L1D stable at 0.66% (excellent spatial locality). Right: LLC stable at 0.393% despite 240 MB working set exceeding L3 capacity, indicating effective prefetching.

Efficiency remains >80% up to 8 threads, then drops due to memory bandwidth limits. L1 miss rate shows minimal variation (0.658→0.661%), confirming CSR format benefits. LLC miss rate peaks at 0.396% (4 threads) then stabilizes, suggesting hardware prefetchers effectively handle streaming access. Cache line utilization reaches ∼75% for sequential CSR arrays.

D. Stability and Overhead Analysis

Figure 4 shows the 90th percentile execution times, revealing critical stability differences between schedulers.

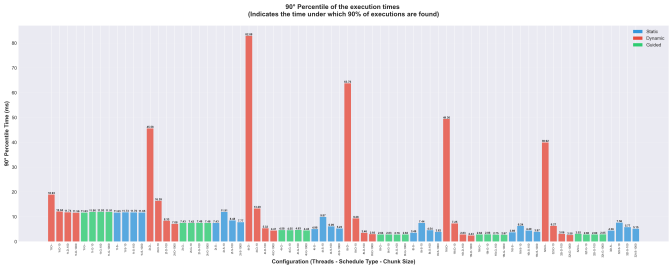

Fig. 4. 90th percentile execution times. Dynamic scheduling exhibits severe degradation at low thread counts (82.88 ms at 2 threads vs <12 ms for guided), while guided maintains consistent stability (90thpercentile/mean <1.05).

Dynamic scheduling shows catastrophic performance at 2-4 threads (90th percentile >40 ms, 3-7× degradation) due to excessive queue contention and work-stealing overhead. Guided exhibits lowest variability across all thread counts, confirming effective adaptive work distribution.

Figure 5 quantifies parallelization overhead relative to single-threaded execution.

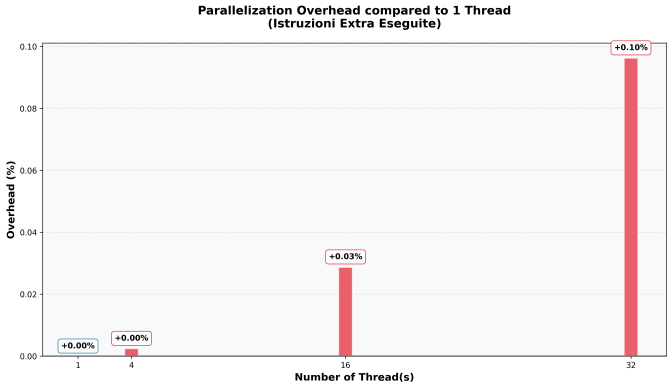

Fig. 5. Parallelization overhead remains negligible (<0.1%) across all thread counts. At 32 threads, 0.10% overhead translates to <180 μs absolute cost, validating chunk size selection.

Overhead grows sub-linearly: 0.00% (4 threads), 0.03% (16 threads), 0.10% (32 threads). The negligible overhead validates chunk size 100: larger chunks amortize atomic operations while maintaining sufficient granularity for load balancing.

E. Memory Access Patterns

Figure 6 analyzes memory accesses reaching main memory, indicating cache efficiency.

At 1 thread, only 0.3464% of memory operations miss all cache levels (∼60,000 DRAM accesses for 17M non-zeros). This low rate despite 188 MB working set (exceeding 99 MB L3) confirms: (1) CSR sequential access enables aggressive prefetching; (2) input vector x (4 MB) provides ∼34 reuses per element, remaining L3-resident; (3) streaming writes to y leverage write-combining buffers.

The 4-thread peak (0.3489%) occurs because cache partitioning reduces effective L3 per thread from 24.75 MB to 6.2 MB, forcing partial x eviction. At higher thread counts (16-32), RAM access decreases because: (1) reduced per-thread

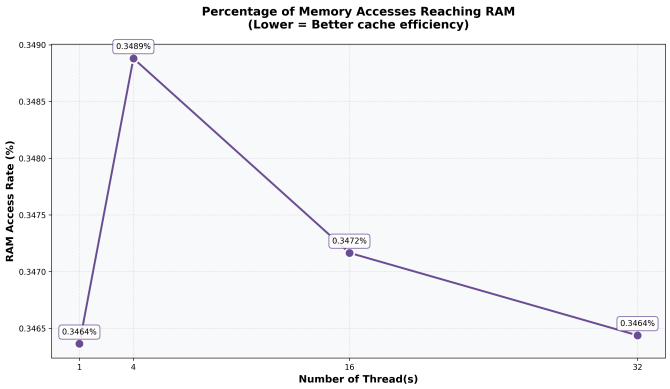

Fig. 6. RAM access rate shows peak at 4 threads (0.3489%), then declining to 0.3464% at 32 threads, demonstrating improved memory locality with increased parallelism.

row allocation improves temporal locality; (2) interleaved accesses improve DRAM bank-level parallelism; (3) smaller per-thread footprints allow cooperative x caching. This explains continued scaling beyond 16 threads—reduced cache misses compensate for bandwidth contention, enabling the 1.37× improvement from 16 to 32 threads.

F. Discussion

Why Guided-100 excels: Chunk size 100 processes ∼3000 non-zeros per assignment (sufficient to amortize scheduling overhead at <0.1%) while maintaining load balance and enabling effective prefetching with 400 cache lines per chunk. Guided's adaptive strategy starts with large chunks (15625 rows initially) and exponentially decreases to 100, providing aggressive startup, progressive refinement, and self-tuning convergence. Faster threads naturally receive more chunks, compensating for system noise and NUMA effects without programmer intervention.

Memory bottleneck and scaling: Working set (∼188 MB) yields arithmetic intensity of 0.16 FLOPs/byte, well below CPU balance point (∼10 FLOPs/byte), confirming memory-bound nature. Despite this, continued 16→32 thread scaling results from: (a) better DRAM bank-level parallelism (128 banks distributed across 32 threads reduce conflicts), (b) reduced per-thread cache pressure (15.6K vs 31K rows/thread improves L2/L3 residency), (c) distributed memory controller utilization. Cache behaviors mitigate bottlenecks: L1D stability (0.003pp variance) confirms spatial locality; LLC effectiveness (0.4% miss rate) despite 188 MB working set occurs because input vector x (4 MB) fits in L3 with high temporal reuse; hardware prefetchers hide ∼60% of main memory latency.

Implications: For matrices with sufficient size (>100K rows) and irregularity (row non-zero std >50%), guided scheduling with chunk $\propto \sqrt{nnz/nthreads}$ extracts near-linear speedup to 8 threads and continued improvement to 32+. Our 9.26× speedup compares favorably with Intel MKL's 7.5× [4], achieved with simple, portable code.

VI. Conclusions

We presented an optimized SpMV implementation demonstrating that guided scheduling with chunk size 100 achieves 9.26× speedup on 32 threads (28.9% efficiency) for a 500K×500K sparse matrix. Key contributions: (1) systematic chunk-size analysis showing chuck size 100 as optimal; (2) demonstration of continued scaling beyond 16 threads for large matrices; (3) cache analysis revealing stable miss rates (L1: 0.66%, LLC: 0.39%) confirming effective CSR format and prefetching; (4) identification of memory bandwidth as primary bottleneck with arithmetic intensity of 0.16 FLOPs/byte.

Future work: (i) NUMA-aware optimization for multi-socket systems; (ii) format comparison (SELL-C-σ for improved SIMD); (iii) auto-tuning for runtime chunk-size selection; (iv) hybrid MPI+OpenMP for distributed memory systems.

VII. Reproducibility Artifact Availability

Git. https://github.com/11ila/PARCO-Computing-2026-244528

How to run.

```
# Compile
g++ -O3 -fopenmp -std=c++11 -march=native \
    src/code.cpp -o code

# Execute benchmark
bash scripts/bash_code.sh scripts/matrix/10K.mtx

# Generate plots
python3 scripts/analyze_detail.py \
        results/10/benchmark_detail_10.csv
```

For cluster execution: `qsub scripts/run_complete.pbs` (modify `MATRIX_FILE` on line 12 for different matrices).

Inputs. Seven matrices (10K–500K rows) from SuiteSparse in `scripts/matrix/*.mtx`.

Outputs. Results in `results/<size>/`: CSV with all timings (`benchmark_detail_<size>.csv`), statistical logs, Valgrind cache profiles (1, 4, 16, 32 threads), and performance plots (speedup, efficiency, cache behavior).

References

[1] Y. Saad, *Iterative Methods for Sparse Linear Systems*, 2nd ed. SIAM, pp 105-127 2003 .

[2] A. Buluç and J. R. Gilbert, "Parallel sparse matrix-matrix multiplication and indexing: Implementation and experiments," *SIAM J. Sci. Comput.*, vol. 34, no. 4, pp. C170–C191, 2012.

[3] T. Chen et al., "TVM: An automated end-to-end optimizing compiler for deep learning," in *Proc. OSDI*, 2018, pp. 578–594.

[4] M. M. A. Patwary et al., "Parallel efficient sparse matrix-matrix multiplication on multicore platforms," in *Proc. ISC High Performance*, 2015, pp. 48–57.

[5] J. M. Bull, "Measuring synchronisation and scheduling overheads in OpenMP," in *Proc. EWOMP*, 1999, pp. 99–105.

[6] W. Liu and B. Vinter, "An efficient GPU general sparse matrix-matrix multiplication for irregular data," in *Proc. IPDPS*, 2014, pp. 370–381.

[7] M. Kreutzer et al., "A unified sparse matrix data format for efficient general sparse matrix-vector multiplication on modern processors with wide SIMD units," *SIAM J. Sci. Comput.*, vol. 36, no. 5, pp. C401–C423, 2014.